Cuttlely



BENCHMARK WHITEPAPER · OCTOBER 2026

Lean context, reliable agents.

Same agent, a third of the tokens, zero failed runs.

A real-world production Flowise deployment at a large enterprise, rebuilt in Cuttlely, Flowise, Langflow and n8n, run on the same model, tools and questions: 36 runs per platform.

3.3x
fewer tokens than Flowise and Langflow; 7.2x fewer than n8n

36 / 36
runs passed on Cuttlely, vs 33, 35 and 24 of 36

$0.34
for all 36 runs, vs $0.71, $0.71 and $2.46 (about 2.1x and 7.2x lower)

INSIDE

1. Executive summary and thesis
2. The problem: context cost
3. Method
4. Results
5. Why fewer tokens means more reliable answers
6. Architecture
7. Limitations
8. Reproduce it
9. Conclusion and references

01 Executive summary

Agents fail in production for a reason that rarely shows up in a demo: their context fills up. Every tool result, every retrieved passage and every copy of a long system prompt is read again on every model call. The bill grows, and the model has more to sift through on every step.

We took an AI assistant workload from a real-world production Flowise deployment at a large enterprise and rebuilt it natively in four platforms: Cuttlely, Flowise, Langflow and n8n. Each side got the same instructions, the same tool set, the same model (OpenAI gpt-4.1-mini at temperature 0.2) and the same 12 questions, each run 3 times in a fresh session. That is 36 runs per platform. Every token count is the usage OpenAI reported on each call.

> **Result.** Cuttlely used **1,924,869 tokens** across 36 runs, against 6,322,010 for Flowise, 6,330,236 for Langflow and 13,843,978 for n8n. That is 30.4% of the Flowise total: **a third of the tokens** (3.3x fewer than Flowise and Langflow, 7.2x fewer than n8n). Cuttlely passed **36 of 36 runs**; the others passed 33, 35 and 24. At list prices the 36 runs cost **$0.34** on Cuttlely, against $0.71, $0.71 and $2.46: about half the cost of Flowise and Langflow and about a seventh of n8n.

The thesis: fewer tokens, cleaner context, more reliable answers

Fewer tokens
Cuttlely carried 53.5K tokens per run, vs 175.6K to 384.6K on the other platforms.
MEASURED

→

Less context bloat
No 194,661-character system prompt re-sent on every call; rules and data load on demand.
MEASURED DESIGN

→

Cleaner context
Published research shows models use long, cluttered contexts less reliably.
RESEARCH

→

More reliable answers
36/36 runs passed. Answer correctness was not graded in this study.
PARTLY MEASURED

Read the chain honestly: the first two links are measured here, the third is established in published research (Section 05), and the last is supported by our completion data, not proven by it. We did not score answer correctness, so we make no accuracy-percentage claim.

What the one-liner means, precisely

- **“Same agent”:** the same production workload, instructions, tool set, model, settings and questions, rebuilt natively in each platform (on Cuttlely the instructions were reorganized into core instructions and playbooks).
- **“A third of the tokens”:** 1.92M vs 6.32M tokens against Flowise (30.4%), and 6.33M against Langflow (30.4%). Against n8n it is about a seventh (13.9%).
- **“Zero failed runs”:** 36 of 36 Cuttlely runs completed and used the expected kind of tool. A pass is a completion check, not a correctness grade.
- **Cost is not a third.** Prompt caching narrows the cost gap: Cuttlely was about 2.1x cheaper than Flowise and Langflow and 7.2x cheaper than n8n.

02 The problem: context is the hidden cost of agents

A typical agent is a loop. The model reads its instructions and the conversation, calls a tool, reads the result, and calls the model again. Graph-based builders make that loop easy to set up: drop an agent node on a canvas, attach a

model, memory and tools, and paste the instructions into one system prompt. What they do not change is what the agent carries. On every call, the whole system prompt and every earlier tool result in the loop are sent again.

The production workload in this study shows why that matters. Its instructions are a single system prompt of **194,661 characters**. In the three graph builders that prompt reached the model on every call, and the average model call carried about **48,000 tokens** on Flowise and Langflow and about **50,000** on n8n (total tokens divided by model calls: 132, 132 and 275 calls).

Cost

Tokens are the unit of the bill. A loop that re-sends a large prompt multiplies it by the number of steps the model takes, which varies by run: one question on Flowise ranged from 46,687 to 245,027 tokens across three runs, and one on n8n from 95,032 to 800,594.

Reliability

A model that has to find one instruction in a 50,000-token prompt, next to several tool results, has more chances to miss it. Published research (Section 05) shows that accuracy drops as context grows and as irrelevant material is added. Long loops are also where runs fail: on n8n, 10 of 36 runs hit the 15-step cap.

Caching lowers the price of a token, not the size of the context. OpenAI served 98.6% of Flowise and Langflow prompt tokens from its prompt cache, 77.0% of n8n's and 82.1% of Cuttlely's. Cached input is billed at a quarter of the normal price, which is why the cost gap is smaller than the token gap. A cached token is still in the context window, and the model still reads all of it on every call.

03 Method

This is a migration-style comparison: one production assistant, rebuilt the way a team would rebuild it in each platform, rather than a synthetic task written for the test. The instructions, tool definitions and questions come from the production deployment. To keep customer data out of the test, the tools were stand-ins that return the same made-up demo data on every side.

ITEM	SETTING
Workload	A real-world production AI assistant (data questions, retrieval questions, a chart, a follow-up turn), from a Flowise deployment at a large enterprise
Questions and runs	12 questions x 3 runs = 36 runs per platform, each run in a fresh session. Q4 is a follow-up turn that continues the Q3 session.
Model	OpenAI gpt-4.1-mini, temperature 0.2, on all four sides
Instructions and tools	The production system prompt (194,661 characters) and the same 11 tool definitions (names, descriptions, arguments); up to 15 model calls or iterations per run
Tool data	Stand-in tools with fixed demo results: a query tool (10 made-up rows), retrieval and search (4 made-up passages), fixed date, code and web results. No real database, vector store, web call, credential or customer data.
Flowise	A replay script that sends the production Flowise Tool Agent requests to OpenAI exactly as Flowise builds them (full system prompt, 11 tool schemas, up to 15 calls). Not a running Flowise server.
Langflow 1.12.6	Real server; Chat Input, Agent, Chat Output; tools through an MCP Tools component; default memory. Built-in calculator and date tools off and temperature set to 0.2 to match.
n8n 2.43.4	Real self-hosted server (official Docker image, community edition); chat trigger, AI Agent, OpenAI Chat Model, Simple Memory, MCP Client Tool. Max iterations raised from 10 to 15 and temperature set to 0.2; everything else default.

ITEM	SETTING
Cuttley	Local server, main branch at commit 70a76d0 plus cached-token reporting. A harness: Harness Prime routes; an analyst specialist has the core instructions, 23 playbooks loaded on demand, code execution and the query tool; a knowledge specialist has search.
Token measurement	OpenAI's reported usage on every call: prompt, cached and output tokens. A local logging proxy for Langflow and n8n (numbers and tool names only); OpenAI usage in the Flowise replay; Cuttley's turn report, including every specialist.
Pass rule	The run completed and used the expected kind of tool: query or code for data questions, retrieval or search for retrieval questions. A run that errored or hit the step cap is a fail, and its tokens are counted in full.
Cost	OpenAI list prices for gpt-4.1-mini per million tokens: $0.40 input, $0.10 cached input, $1.60 output. Uncached input = prompt minus cached.

The Langflow and n8n runs were completed on 13 October 2026, using the latest stable release of each that day. The Flowise and Cuttley figures come from the same benchmark and were not re-run when Langflow and n8n were added.

04 Results

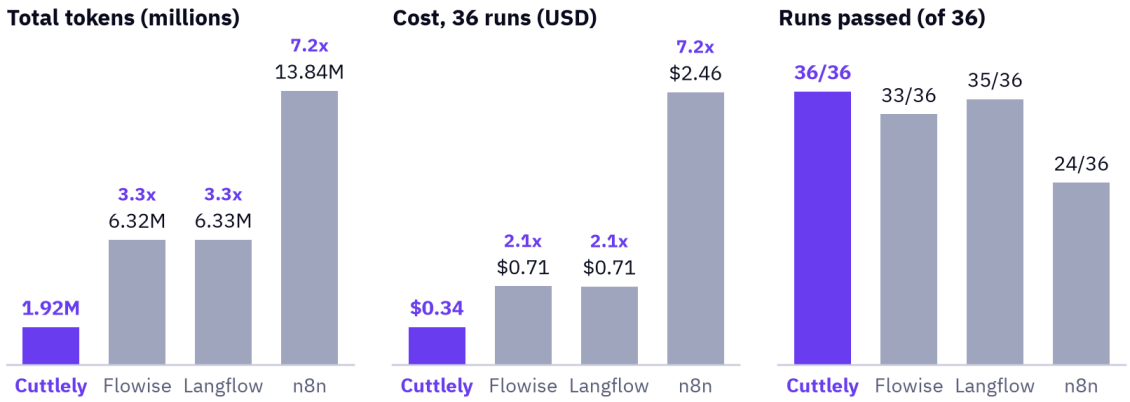


Figure 1. Total tokens (prompt + output, as reported by OpenAI), cost at gpt-4.1-mini list prices with the cached-input discount, and runs passed, for 36 runs per platform. Violet labels are each platform's ratio to Cuttley, from unrounded values. A pass is a completion check, not a correctness grade.

PLATFORM	PROMPT TOKENS	CACHED	OUTPUT TOKENS	TOTAL TOKENS	MODEL CALLS	COST	COST, NO CACHE	PASSED
Cuttley	**1,891,489**	**82.1%**	**33,380**	**1,924,869**	**162 steps***	**$0.34**	**$0.81**	**36/36**
Flowise	6,287,110	98.6%	34,900	6,322,010	132	$0.71	$2.57	33/36
Langflow 1.12.6	6,297,602	98.6%	32,634	6,330,236	132	$0.71	$2.57	35/36
n8n 2.43.4	13,758,036	77.0%	85,942	13,843,978	275	$2.46	$5.64	24/36

* Cuttley's turn report counts tool steps across Harness Prime and its specialists, not model calls, so its count is not directly comparable. Without any cache discount the cost gap would be 3.2x against Flowise and Langflow and 7.0x against n8n.

Per question

Cuttley had the lowest mean tokens on all 12 questions, and the margin was clear on 11. The near-tie is Q10, a question that needs both data and retrieval, where Cuttley used both specialists and about 10 tool steps: 96,108 tokens against 97,047 for Flowise and 96,131 for Langflow (1.0x) and 131,786 for n8n (1.4x). The largest gap was Q12, a retrieval-only question: 7,340 tokens against 78,549, 94,340 and 126,626.

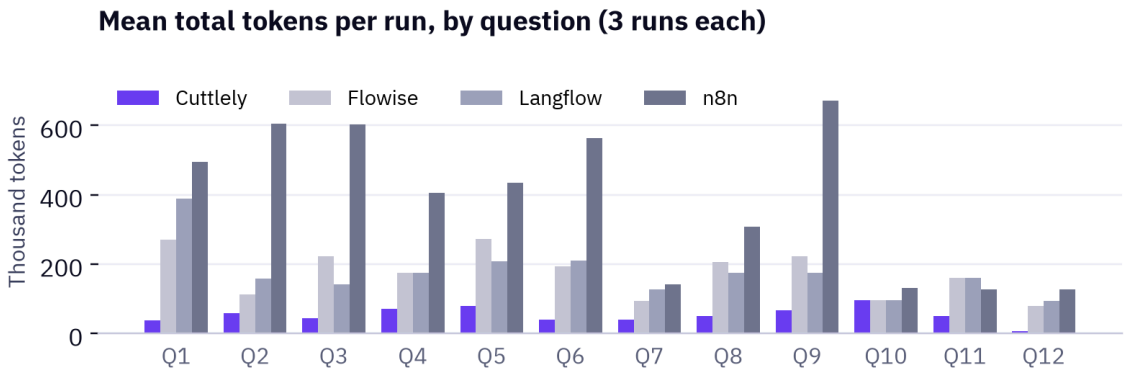


Figure 2. Mean total tokens per run for each question (3 runs each).

Where the failures were. Flowise failed twice on Q2 and once on Q12. Langflow's one fail (Q4, run 3) answered the follow-up from memory without calling a tool. n8n's 12 fails were 10 runs that hit the 15-step cap while re-running the query tool, mostly with SQL it had already run, plus 2 runs on Q7 that used retrieval but no query.

05 Why fewer tokens means more reliable answers

Cost is the easy argument for a smaller context. Reliability is the more important one.

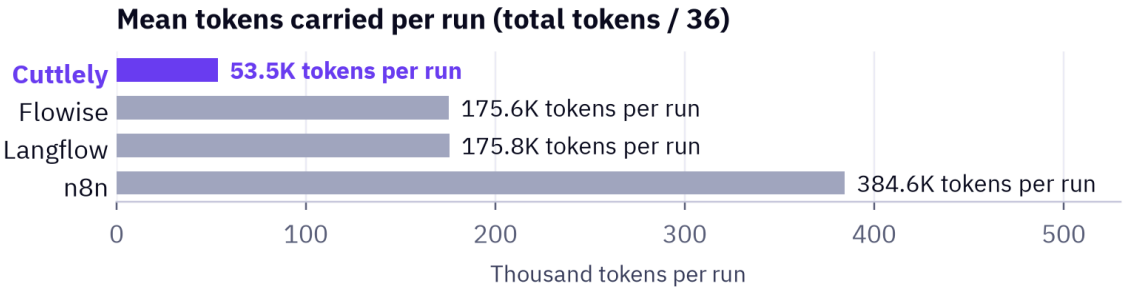


Figure 3. Mean tokens per run (total tokens divided by 36). Every one of these tokens is read by the model, cached or not.

What the research shows

- **Position and length matter.** Liu et al. found that performance is often highest when the relevant information is at the start or end of the input and degrades significantly when it sits in the middle of a long context, even for models built for long contexts [1].
- **Irrelevant context distracts.** Shi et al. showed that adding irrelevant information to a problem description dramatically lowers accuracy on grade-school math problems [2].
- **Advertised context is not usable context.** On the RULER benchmark, almost all of the 17 long-context models tested showed large drops as context length increased, and only half held satisfactory performance at 32K tokens [3].
- **It holds for current models.** Chroma's Context Rot report tested 18 models, including GPT-4.1, and found that performance grows increasingly unreliable as input length grows, even on simple tasks [4].

What our data adds

- Cuttlely's runs carried 53.5K tokens on average, against 175.6K, 175.8K and 384.6K. In the graph builders, the average model call alone carried about 48,000 to 50,000 tokens, above the 32K length at which only half of the models tested in [3] held satisfactory performance.

- The leanest platform passed every run (36/36). The platform with the largest context per run, n8n, passed the fewest (24/36), and most of its fails were loops that re-ran a query it had already run instead of answering from the rows it had. The pattern is not strictly ordered: Flowise and Langflow carried almost identical context and passed 33 and 35. We read it as consistent with the mechanism, not as proof of it.
- Demo tool results here were small (10 rows, 4 passages). Real query results and documents are larger, and in a single-agent loop each one is re-sent on every later call and is not yet cached. The gap is likely to grow on real data, not shrink. This is a reasoned expectation, not a measurement.

> **What we claim and what we don't.** We measured tokens, cost and completion. We did not grade the correctness of answers: the demo data cannot answer the production questions, so a correctness score would be meaningless here. The link from a smaller context to more accurate answers rests on the published research above and on the completion data, and we present it as a mechanism, not a measured accuracy gain.

06 Why Cuttlely uses fewer tokens: the architecture



In a graph builder, the structure of the agent is fixed when you draw it: what each node sees and which prompt it carries are wired in advance, so the safe choice is to give the agent everything, every time. Cuttlely runs specialists on a real agent runtime, Hermes from Nous Research, which ships with Cuttlely and starts with it. The runtime decides its path at run time: which specialist works on which part of the question, which playbook to open, and what to hand back.

Each specialist gets its own task

Harness Prime reads the question and writes one self-contained task per specialist, with a short recap of recent turns, instead of handing every agent the whole chat. A specialist's instructions live in its runtime profile and are not re-sent with each task.

Rules load when they're needed

Long instructions go into playbooks. A specialist sees each playbook's name and when-to-use line, and opens the content only when a question needs it. In this benchmark the production prompt became core instructions plus 23 playbooks.

Data stays in the database

The analyst returns one page of rows (100 by default, configurable from 1 to 1,000). Earlier pages stay out of the next prompt.

Short, capped handoffs

Harness Prime reads a capped part of each specialist's result (4,000 characters by default) with source names and short snippets; full documents go back to your app in the response. By default the specialist prompt is capped at 4,000 tokens and the handoff prompt at 6,000.

Built for production changes, not just first runs

- **Draft, compare, publish, roll back.** Every edit saves to a draft while Live keeps serving your app. Compare replays recent questions on Live and on the draft and marks each answer Same, Changed or Error; every version is kept and Live rolls back in one step.
- **A/B test with a passing test set.** Score the draft with text, JSON, numeric and model evaluators on up to 100 question-and-answer pairs, and make Publish wait until it passes; `cuttlely check --gate` runs the same check in CI.
- **See every token.** The run card and the API response show input, output and cached tokens per step. OpenTelemetry traces for each chatflow prediction, agentflow run and harness run can go to your own collector.
- **Bring your existing flows.** Import Flowise JSON and Import Langflow JSON open existing flows on the Cuttlely canvas.

Credits: Cuttlely's canvas editor and workflow engine come from Flowise (Apache-2.0), and its agent runtime is Hermes from Nous Research (MIT). See CREDITS.md in the repository.

07 Limitations and threats to validity

- **One workload, one model.** Results come from a single production assistant on gpt-4.1-mini. The production flow is configured for a different model; absolute tokens and prices will shift on other models. Treat the ratios, not the totals, as the finding.
- **Stand-in tools and small data.** Tools returned fixed, made-up results. Real results are larger, which we expect to widen the gap for single-agent loops, but this was not measured.
- **Completion, not correctness.** A pass means the run completed and used the expected kind of tool. Answer quality was not graded.
- **Harness differences.** The Flowise side replays the production flow's requests to OpenAI as Flowise builds them, so no Flowise server version applies. Langflow and n8n reached the tools through MCP stand-ins with matching names, descriptions and arguments, not their native database, retrieval or code components.
- **Defaults and tuning.** For n8n only max iterations and temperature were changed. Its default Responses API mode cached less: a one-run check with Chat Completions cached 95.9% and cost $0.59 for the 12 questions vs $0.82 on average with the default, at about the same tokens, and 2 of 12 runs still hit the cap. The headline uses n8n's defaults.
- **Different architectures.** Cuttlely ran the workload as a harness with specialists and playbooks; the others ran it as a single agent, as the production flow does. That difference is the subject of the study, not a confound, but readers should know the Cuttlely build was restructured, not a one-click import.
- **Variance.** At temperature 0.2 single questions swing. Cuttlely's Q10 ranged from 78,689 to 130,612 tokens. A previous Cuttlely run on an earlier commit used 1,750,200 tokens, 10% fewer than the 1,924,869 reported here; we report the higher figure.
- **Caching depends on traffic.** OpenAI caches prompts of 1,024 tokens or more for a few minutes. Back-to-back test traffic may cache more or less than production.
- **Build vs release.** The Cuttlely build was main at commit 70a76d0 plus cached-token reporting, not the tagged 1.2.0 npm release; numbers may differ slightly on the release.
- **Who ran it.** The benchmark was run by the Cuttlely team. Independent reproduction is welcome.

08 Reproduce it

The production instructions and questions belong to the enterprise that runs them, so they are not published. The method is fully reproducible with your own agent:

1. **Pick one real workload.** Use an existing agent's system prompt, tool definitions and 10 to 15 real questions, including a follow-up turn.
2. **Fix the variables.** One model and temperature on every side (we used gpt-4.1-mini at 0.2), the same tool names, descriptions and arguments, and the same iteration cap (15).
3. **Use stand-in tools.** Serve the tools from one MCP server that returns identical fixed results to every platform, so data size and content cannot differ.
4. **Build natively.** Flowise or Langflow: an agent node with the prompt and tools. n8n: chat trigger, AI Agent, OpenAI Chat Model, Simple Memory, MCP Client Tool. Cuttlely: build a harness (Build with AI can draft one) and move long rules into playbooks; Import Flowise JSON brings an existing flow over as a starting point.
5. **Measure at the provider.** Put a logging proxy between each platform and the model API and record only the reported usage (prompt, cached, output tokens) per call. Cuttlely's run card and API response report the same

fields.

6. **Run 3x per question** in fresh sessions, apply the pass rule above, count failed runs' tokens in full, and price with the provider's list prices.

09 Conclusion

On a real production workload, rebuilt natively in four platforms, Cuttlely did the same job with **a third of the tokens** of Flowise and Langflow and a seventh of n8n, passed **all 36 runs**, and cost **about half** as much as Flowise and Langflow and a seventh as much as n8n. The reason is architectural: a runtime that decides at run time what each step needs keeps the context lean, and published research shows that models use shorter, cleaner contexts more reliably.

Try it on your own workload

Repository: github.com/cuttlely/cuttlely · Website: cuttlely.com · npm: cuttlely (1.2.0, Apache-2.0)
Docs: Context and tokens · Harnesses · A/B testing · Observability · Bring your existing flows

References

1. N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang. “Lost in the Middle: How Language Models Use Long Contexts.” *Transactions of the ACL* 12:157–173, 2024. arXiv:2307.03172 (2023). aclanthology.org/2024.tacl-1.9
2. F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. Chi, N. Schärli, D. Zhou. “Large Language Models Can Be Easily Distracted by Irrelevant Context.” *ICML 2023*, PMLR 202:31210–31227. arXiv:2302.00093
3. C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, B. Ginsburg. “RULER: What's the Real Context Size of Your Long-Context Language Models?” *COLM 2024*. arXiv:2404.06654
4. K. Hong, A. Troynikov, J. Huber. “Context Rot: How Increasing Input Tokens Impacts LLM Performance.” Chroma technical report, July 2025. research.trychroma.com/context-rot
5. OpenAI. gpt-4.1-mini model pricing ($0.40 input, $0.10 cached input, $1.60 output per 1M tokens). developers.openai.com

Benchmark data: Cuttlely side-by-side token benchmark, runs completed 13 October 2026. Flowise, Langflow and n8n are trademarks of their respective owners and are named here only to identify the platforms tested. This paper was produced by the Cuttlely team.